

ESTUDO TÉCNICO APLICADO DO PROTOCOLO HTTP

Aplicação no Portal Educacional EAD, com análise de requisições, autenticação, arquivos e segurança por HTTPS

Tema central

O HTTP é estudado como protocolo de camada de aplicação. O HTTPS aparece como requisito de segurança para uma implantação real do portal, sem substituir o foco principal do relatório.

Acadêmico: Miguel Montemezzo

Relatório público de estudo aplicado de protocolos

Resumo

Este relatório apresenta um estudo técnico aplicado do Hypertext Transfer Protocol (HTTP) no Projeto Integrador “Portal Educacional EAD”. O objetivo é demonstrar como o protocolo participa do funcionamento real do sistema, ligando o navegador à API desenvolvida em Node.js e Express. A fundamentação utiliza a RFC 9110, que define a semântica do HTTP, além das RFCs 8446 e 6750 para discutir TLS/HTTPS e o transporte de tokens do tipo Bearer. A parte aplicada examina o repositório público do projeto, especialmente o módulo que centraliza chamadas com Fetch, as rotas do servidor, o fluxo de login, a proteção por token, o tratamento de códigos de estado e as operações de envio e download de arquivos. Como o sistema foi construído para execução local, a aplicação operacional é apresentada por análise direta da implementação e por um roteiro reproduzível com navegador e cURL. Conclui-se que o HTTP é a base de integração entre interface e backend, enquanto o uso de HTTPS é indispensável em produção para proteger credenciais, tokens, notas e arquivos educacionais.

Palavras-chave: HTTP; HTTPS; API web; Express; autenticação Bearer; Projeto Integrador.

1. Introdução

Sistemas web dependem de protocolos para que clientes e servidores interpretem solicitações e respostas de modo compatível. No Portal Educacional EAD, o navegador precisa solicitar páginas, autenticar usuários, consultar turmas, publicar atividades, registrar notas e transferir arquivos. Essas ações são realizadas por mensagens HTTP dirigidas a rotas específicas da aplicação.

O objetivo deste estudo é identificar, explicar e analisar o uso do HTTP no Projeto Integrador, relacionando conceitos normativos com decisões presentes no código. O trabalho não trata o HTTP apenas como “acesso a um site”: examina métodos, recursos, cabeçalhos, corpos em JSON ou multipart, códigos de estado, autenticação e comportamento do cliente diante de erros.

A metodologia combina pesquisa documental em padrões da IETF e documentação oficial do Express com inspeção técnica do repositório público do projeto. A aplicação operacional é reproduzível localmente, pois o repositório fornece instruções de instalação, banco MySQL e inicialização do servidor em `http://localhost:3000` [5].

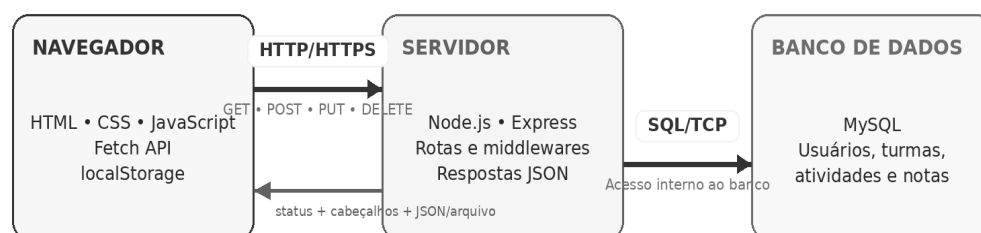
2. Caracterização do Projeto Integrador

O Portal Educacional EAD é uma aplicação acadêmica inspirada em ambientes virtuais de aprendizagem. O sistema possui áreas para alunos, professores, administradores e coordenadores, com funções de autenticação, matrículas, materiais, atividades, envios, notas e avisos. O frontend é escrito em HTML, CSS e JavaScript; o backend utiliza Node.js e Express; e a persistência ocorre em MySQL [5].

A arquitetura segue o modelo cliente-servidor. O navegador carrega os arquivos estáticos e executa scripts que chamam a API. O servidor Express interpreta o método e o caminho da requisição, executa middlewares de segurança e permissão, consulta o banco quando necessário e devolve uma resposta HTTP. O banco não é acessado diretamente pelo navegador: ele permanece atrás da API.

Arquitetura de comunicação do Portal Educacional EAD

O HTTP conecta a interface do navegador à API Express; a API acessa o MySQL internamente.



3. Fundamentação técnica do HTTP

3.1 Natureza e modelo de comunicação

A RFC 9110 define o HTTP como um protocolo de camada de aplicação, sem estado, destinado a sistemas de informação distribuídos [1]. “Sem estado” significa que cada requisição deve conter as informações necessárias para ser compreendida; a continuidade de uma sessão é construída pela aplicação, por exemplo, com um token enviado a cada chamada protegida.

A unidade lógica do protocolo é o ciclo requisição–resposta. A requisição identifica um recurso por URI, informa um método e pode incluir cabeçalhos e conteúdo. A resposta contém um código de estado, cabeçalhos e, quando aplicável, uma representação do recurso. No PI, essas representações são predominantemente JSON, HTML, CSS, JavaScript e arquivos para download.

3.2 Métodos utilizados

O método expressa a intenção do cliente. A RFC 9110 determina que ele é a principal fonte da semântica da requisição [1]. O Portal usa uma interface orientada a recursos, em que caminhos como /alunos, /turmas, /atividades e /notas são combinados com métodos HTTP.

Método	Semântica no HTTP	Exemplo no Portal
GET	Obter uma representação do recurso.	GET /turmas/minhas; GET /auth/me
POST	Processar dados e, frequentemente, criar um recurso.	POST /auth/login; POST /atividades
PUT	Substituir ou atualizar a representação definida pela aplicação.	PUT /alunos/:id; PUT /materiais/:id
DELETE	Solicitar a remoção do recurso.	DELETE /avisos/:id
PATCH	Atualização parcial; suportada pelo cliente genérico.	Disponível em public/js/api.js, ainda que não seja central nas rotas analisadas.

3.3 Cabeçalhos, conteúdo e estados

O cabeçalho Content-Type descreve o formato do conteúdo. O módulo do frontend define application/json para objetos enviados à API e serializa o corpo com JSON.stringify. Para upload, o navegador usa FormData e gera multipart/form-data com o limite adequado, sem que o código force manualmente o cabeçalho [6].

Os códigos de estado resumem o resultado. A implementação usa, entre outros, 200 para sucesso, 201 para criação, 400 para dados inválidos, 401 para token ausente ou inválido, 403 para usuário sem permissão, 404 para recurso ou rota inexistente, 409 para conflitos de integridade e 500 para falhas internas ou de banco [7–9]. Essa distinção permite que o frontend apresente mensagens coerentes e tome ações específicas, como limpar a sessão após um 401.

HTTP não é o mesmo que REST

REST é um estilo arquitetural. O protocolo observado é HTTP; o projeto utiliza uma API orientada a recursos e aproveita métodos e códigos HTTP, mas o relatório não depende de classificar toda a API como “REST pura”.

4. Aplicação do HTTP na implementação

4.1 Cliente: centralização das requisições

O arquivo public/js/api.js funciona como uma camada comum entre as telas e o servidor. Ele monta a URL, obtém o token salvo no navegador, adiciona cabeçalhos, converte objetos em JSON, executa fetch e interpreta a resposta conforme o Content-Type. A mesma função oferece operações get, post, put, patch e delete [6].

```
const resposta = await fetch(montarUrl(caminho), config);
const dados = contentType.includes("application/json")
  ? await resposta.json()
  : await resposta.text();

if (resposta.status === 401) {
  window.Session.limparSessao();
  throw new Error("Sessão expirada.");
}
```

Quadro 1 – Comportamento resumido do cliente HTTP do projeto [6].

Esse desenho reduz repetição e padroniza o tratamento de falhas. As telas de aluno e professor chamam funções de alto nível, como `Api.get("/materiais")` ou `Api.post("/notas", dados)`, enquanto a camada comum cuida dos detalhes de transporte.

4.2 Servidor: rotas e middlewares

No backend, `src/app.js` configura o processamento de JSON, formulários URL-encoded, arquivos estáticos, CORS, cabeçalhos de segurança com Helmet e limitação de tentativas no login. Em seguida, associa prefixos de URI a conjuntos de rotas. O middleware de autenticação é aplicado antes das rotas de alunos, professores, turmas, materiais, atividades, notas, avisos e arquivos [7].

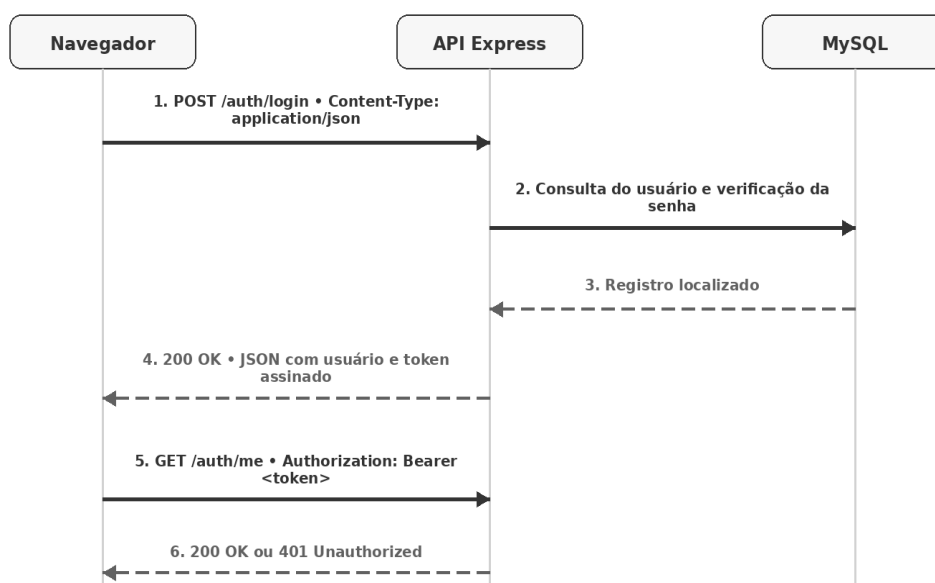
Elemento HTTP	Implementação no PI	Efeito operacional
URI/rota	<code>/auth</code> , <code>/alunos</code> , <code>/turmas</code> , <code>/materiais</code> etc.	Seleciona o recurso e o controlador.
Método	<code>router.get</code> , <code>post</code> , <code>put</code> e <code>delete</code>	Define a ação aceita para cada rota.
Middleware	autenticar e permitir(perfis)	Pode interromper o ciclo com 401 ou 403.
Corpo JSON	<code>express.json({ limit: "1mb" })</code>	Converte a representação recebida para <code>req.body</code> .
Resposta	<code>res.status(...).json(...)</code> ou <code>res.download(...)</code>	Envia estado, cabeçalhos e conteúdo ao cliente.

4.3 Autenticação com Bearer

O login recebe identificador e senha por `POST /auth/login`. Se os dados forem válidos, o servidor responde com o usuário e um token. Nas chamadas posteriores, o cliente envia `Authorization: Bearer <token>`. Esse formato segue o esquema sintático descrito pela RFC 6750 para transporte de credenciais Bearer em cabeçalho HTTP [3].

Um ponto técnico importante é que o token do projeto não é um JWT padrão. O arquivo `src/utlis/tokens.js` cria um payload Base64URL e uma assinatura HMAC-SHA-256, resultando em duas partes separadas por ponto. Um JWT assinado normalmente possui três partes e estrutura definida pela RFC 7519 [4]. Portanto, o termo correto neste relatório é “token próprio assinado”, transportado no esquema Bearer [8].

Fluxo HTTP de autenticação e acesso protegido



5. Aplicação prática e operacional

Como o projeto foi concebido para rodar localmente e não possui uma instância pública indicada no repositório, a prática foi estruturada de duas formas complementares: (a) inspeção da implementação publicada; e (b) roteiro reproduzível de observação das mensagens HTTP. O procedimento não exige alterar o sistema.

5.1 Preparação do ambiente

1. Executar o script database/DB.sql no MySQL e configurar as variáveis do arquivo .env.
2. Na raiz do projeto, executar npm install e npm start.
3. Abrir http://localhost:3000 no navegador. O Express serve o frontend e a API na mesma origem.
4. Pressionar F12 e abrir a aba Rede/Network, preservando o log durante o login e a navegação.

5.2 Casos de teste reproduzíveis

Teste	Requisição	Resultado esperado pela implementação
Disponibilidade	GET /health	200 OK com JSON: status “online” e nome do serviço.
Login válido	POST /auth/login com JSON	200 OK, mensagem, usuário e token.
Login incompleto	POST /auth/login sem senha	400 Bad Request.
Rota protegida sem token	GET /auth/me	401 Unauthorized com orientação sobre Bearer.
Rota protegida com token	GET /auth/me + Authorization	200 OK com dados do usuário.
Rota inexistente	GET /rota-que-nao-existe	404 Not Found em JSON.
Excesso de tentativas	Repetir login além do limite configurado	429 Too Many Requests pelo rate limiter.
Upload aceito	POST /arquivos com multipart/form-data	201 Created com metadados do arquivo.
Arquivo inválido/grande	POST /arquivos fora das regras	400 Bad Request.

5.3 Execução por cURL

```
curl -i http://localhost:3000/health
```

```
curl -i -X POST http://localhost:3000/auth/login \  
-H "Content-Type: application/json" \  
-d '{"login":"admin","senha":"123456"}'
```

```
curl -i http://localhost:3000/auth/me \  
-H "Authorization: Bearer SEU_TOKEN_AQUI"
```

A opção -i exibe a linha de estado e os cabeçalhos. No DevTools, os mesmos elementos aparecem separados em General, Request Headers, Payload, Response Headers e Response. Assim, é possível comprovar operacionalmente método, URI, status, Content-Type, Authorization e corpo da mensagem.

5.4 Transferência de arquivos

O Portal também demonstra que HTTP não transporta apenas JSON. O endpoint POST /arquivos recebe multipart/form-data, valida extensão, MIME e tamanho, e responde 201 com metadados. O endpoint GET /arquivos/:id/download verifica permissão e usa uma resposta de download. Isso evidencia a flexibilidade das representações HTTP e a importância dos cabeçalhos de tipo e disposição de conteúdo [9].

6. Segurança, limitações e melhorias

6.1 HTTP local e necessidade de HTTPS

No ambiente de apresentação, o servidor é anunciado como `http://localhost:3000`. Em uma única máquina, isso facilita o desenvolvimento. Porém, uma implantação acessível por rede não deve transmitir login, senha ou token em HTTP aberto. O TLS cria um canal seguro e fornece confidencialidade, integridade e autenticação do servidor [2]. Quando HTTP opera sobre TLS, utiliza-se o esquema HTTPS.

A própria RFC 6750 recomenda TLS para evitar divulgação e reutilização de tokens Bearer, pois qualquer parte que obtenha um token válido pode tentar usá-lo [3]. Isso é especialmente relevante no portal, que trata dados pessoais, notas, materiais e arquivos de atividades.

6.2 Controles já presentes

Controle	Como aparece no código	Avaliação
Hash de novas senhas	scrypt com salt aleatório.	Boa base para armazenamento de credenciais.
Token com expiração	Assinatura HMAC e validade padrão de 8 horas.	Impede alteração silenciosa e limita a sessão.
Autorização por perfil	Middleware permitir(...) por rota.	Reduz acesso indevido entre perfis.
Rate limiting	Limite de tentativas em <code>/auth/login</code> .	Ajuda contra força bruta.
Validação de upload	Extensão, MIME, tamanho e nome aleatório.	Reduz arquivos perigosos e colisões.
Helmet e CORS	Cabeçalhos de segurança e origem configurável.	Úteis, mas exigem configuração de produção.

6.3 Pontos de atenção

- O Content Security Policy está desativado na configuração do Helmet. Em produção, deve ser planejado e ativado para reduzir o impacto de injeções de script.
- O token e os dados do usuário são armazenados em `localStorage`. Essa escolha é simples, porém um XSS poderia ler o token; por isso, prevenção de XSS, CSP e HTTPS tornam-se essenciais.
- Existe compatibilidade temporária com senhas antigas em texto puro. Essa exceção deve ser removida após migração de todos os registros para hash.
- O token é um formato próprio. Embora assinado, uma solução padronizada e amplamente auditada pode simplificar interoperabilidade, renovação, revogação e revisão de segurança.
- CORS usa “*” por padrão no ambiente. Em produção, a origem deve ser restrita ao domínio real do portal.

Achado principal de segurança

HTTPS não é um “extra visual”. Ele é a condição para que o login e o cabeçalho `Authorization: Bearer` atravessem uma rede sem exposição direta a observadores do caminho.

7. Recomendações e conclusão

7.1 Recomendações para implantação

1. Publicar o portal somente por HTTPS, com certificado válido e redirecionamento permanente de HTTP para HTTPS.
2. Definir `CORS_ORIGIN` para o domínio autorizado e manter o backend inacessível por origens não necessárias.

3. Ativar uma política CSP compatível com os scripts e estilos do projeto, eliminando dependências inseguras antes da ativação.
4. Migrar todas as senhas legadas para script e retirar a comparação com texto puro.
5. Rever a estratégia de sessão: validade curta, mecanismo de renovação ou revogação e avaliação entre token em memória, armazenamento web e cookie HttpOnly/Secure conforme o modelo de ameaça.
6. Documentar formalmente a API, incluindo método, caminho, autenticação, corpo, respostas e códigos de erro. Uma especificação OpenAPI seria apropriada.
7. Registrar evidências da prática no repositório: capturas da aba Network, comandos cURL e respostas sem expor tokens ou dados reais.

7.2 Conclusão

O estudo demonstra que o HTTP é um componente estrutural do Portal Educacional EAD. Ele organiza a comunicação entre navegador e servidor por meio de métodos, URIs, cabeçalhos, conteúdos e códigos de estado. A implementação não utiliza o protocolo apenas para entregar páginas: o HTTP transporta autenticação, consultas, cadastros, atualizações, exclusões, erros, arquivos e respostas do sistema.

A análise do código confirmou uma aplicação concreta dos conceitos da RFC 9110. O frontend centraliza fetch e tratamento de respostas; o Express associa métodos a rotas; os middlewares controlam autenticação e perfis; e os controladores devolvem estados coerentes. O fluxo de login mostra também como um token próprio é enviado no cabeçalho Authorization com esquema Bearer.

Para desenvolvimento e apresentação local, o uso de `http://localhost` é operacionalmente adequado. Para uma versão disponibilizada a usuários reais, entretanto, HTTPS é obrigatório do ponto de vista técnico, pois o portal transmite credenciais e informações educacionais sensíveis. O protocolo HTTP permanece o mesmo em sua semântica, mas passa a operar dentro do canal protegido pelo TLS.

Assim, a relação entre o estudo e o Projeto Integrador é direta e verificável: cada funcionalidade relevante do portal depende de uma troca HTTP. A prática proposta permite observar essas trocas sem construir uma aplicação paralela, aproveitando o próprio PI como laboratório de protocolos.

Referências

- [1] FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. RFC 9110: HTTP Semantics. IETF, 2022. <https://www.rfc-editor.org/rfc/rfc9110> Acesso em: 2 jul. 2026.
- [2] RESCORLA, E. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. IETF, 2018. <https://www.rfc-editor.org/rfc/rfc8446> Acesso em: 2 jul. 2026.
- [3] JONES, M.; HARDT, D. RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage. IETF, 2012. <https://www.rfc-editor.org/rfc/rfc6750> Acesso em: 2 jul. 2026.
- [4] JONES, M.; BRADLEY, J.; SAKIMURA, N. RFC 7519: JSON Web Token (JWT). IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7519> Acesso em: 2 jul. 2026.
- [5] BALDI, P. et al. PI5 – Portal Educacional EAD: repositório público e README. GitHub, 2026. <https://github.com/pedrobaldi01/PI5-PortalEducacionalEAD> Acesso em: 2 jul. 2026.
- [6] PI5 – Portal Educacional EAD. public/js/api.js, auth.js e session.js. GitHub, 2026. <https://github.com/pedrobaldi01/PI5-PortalEducacionalEAD/tree/main/public/js> Acesso em: 2 jul. 2026.
- [7] PI5 – Portal Educacional EAD. src/app.js, rotas e middleware de erros. GitHub, 2026. <https://github.com/pedrobaldi01/PI5-PortalEducacionalEAD/tree/main/src> Acesso em: 2 jul. 2026.
- [8] PI5 – Portal Educacional EAD. auth.controller.js, auth.middleware.js e tokens.js. GitHub, 2026. <https://github.com/pedrobaldi01/PI5-PortalEducacionalEAD/tree/main/src> Acesso em: 2 jul. 2026.
- [9] PI5 – Portal Educacional EAD. arquivos.routes.js, arquivos.controller.js e upload.middleware.js. GitHub, 2026. <https://github.com/pedrobaldi01/PI5-PortalEducacionalEAD/tree/main/src> Acesso em: 2 jul. 2026.
- [10] EXPRESS.JS. Routing – Express web framework documentation. <https://expressjs.com/en/guide/routing/> Acesso em: 2 jul. 2026.
- [11] EXPRESS.JS. Using middleware – Express web framework documentation. <https://expressjs.com/en/guide/using-middleware/> Acesso em: 2 jul. 2026.